

Linux Fundamentals: My Review Notes

Everything covered on my Kali VM (user ajay360). Quiz score: 18/20.

How I learn this

Predict what a command will do **before** pressing Enter. Run it. Read the output slowly, especially errors. Compare with the prediction. Copying commands without understanding them is not learning.

Read-only commands first. Never run an unknown script blindly (for example `wazuh-install.sh`).

1. Navigation

Command	What it does
<code>pwd</code>	Print the folder I am in.
<code>ls</code>	List what is in the folder.
<code>ls -a</code>	Include hidden files (names starting with a dot).
<code>ls -l</code>	Long format: permissions, owner, group, size, date, name.
<code>cd folder</code>	Go into a folder.
<code>cd ..</code>	Go up one level only.
<code>cd ~</code>	Jump straight home from anywhere. Say it "cd tilde".

Symbol	Meaning
<code>~</code>	My home folder, <code>/home/ajay360</code>
<code>.</code>	The folder I am in right now (so <code>./script.sh</code> means the script in this folder)
<code>..</code>	The folder one level up

2. Files and folders

```
mkdir lab           # make a folder
touch testperm.txt  # make an empty file
cp a.txt b.txt      # copy: two files afterward
mv b.txt c.txt      # move/rename: one file afterward
mv c.txt lab/       # move into a folder (slash makes intent clear)
rm c.txt            # delete a file
rmdir lab           # delete an EMPTY folder only
rm -r lab           # delete a folder and everything in it
```

rm -r has no warning and no undo. `rmdir` refusing ("Directory not empty") was a safety net, and `rm -r` takes it away.

mv gotcha: if the destination folder does not exist, `mv file backup` renames the file to backup instead. Check with `ls -l`: a first character of `-` is a file, `d` is a folder. Renaming keeps the file's permissions.

cp gotcha: `cp` alone only copies files. Copying a folder needs `-r` (recursive). Not tested yet.

3. Reading files, redirection, errors

```
cat file.txt          # show a text file
echo "hello" > file.txt # > OVERWRITES the file
echo "more" >> file.txt # >> APPENDS to the file
```

cat is for text. On a binary file such as a .tar archive it dumps garbage and can scramble the terminal. Type reset and press Enter to fix it. To look inside an archive: tar -tf file.tar.

Errors are teachers. "No such file or directory" was one wrong letter (note.text vs note.txt). "Permission denied" means the permissions block me. "Directory not empty" is rmdir protecting me. Linux does not care what I name things, only that names match exactly.

4. Searching with find

ls shows one folder. find digs through folders and subfolders and prints the full path.

```
find ~ -name "testperm.txt" # search home for an exact name
find ~ -type d              # every folder under home (long list)
find ~ -type f              # every regular file
```

5. Reading permissions

Example: -rwxr-xr-- is ten characters.

Part	Example	Meaning
Type	-	- file, d directory
Owner	rwx	what the owner can do
Group	r-x	what the group can do
Others	r--	what everyone else can do

r read, w write, x execute, - that permission is off. Always in the order r, w, x.

Execute on a file means run it. **Execute on a folder** means enter it (cd into it). Read on a folder lets me list its names. Without x on wazuh-install.sh (rw-rw-r--), ./wazuh-install.sh would say "permission denied".

Check order: Linux checks owner first, then group, then others. Only the first match applies. I am ajay360, so wazuh-install-files.tar (owner root, rw-----) gave "Permission denied". Deleting a file depends on the **folder's** permissions, not the file's.

6. chmod: changing permissions

Letters

Who: u owner, g group, o others, a all. Then + or -, then r, w, x.

```
chmod u+x testperm.txt # owner gets execute -rw-rw-r-- -> -rwxrw-r--
chmod g+w testperm.txt # group gets write
chmod go-r testperm.txt # remove read from group and others
chmod +x file           # no letter before + gives execute to everyone
```

Numbers

r = 4, w = 2, x = 1, off = 0. Add them **within** each group. Write the three digits **side by side**, never add them together.

Number	Result	Working
754	rwxr-xr--	7 = 4+2+1, 5 = 4+1, 4 = 4
640	rw-r-----	6 = 4+2, 4 = 4, 0 = none
750	rwxr-x---	owner full, group read+execute, others nothing
720	rwx-w----	7, 2, 0

Why chmod 777 is a bad habit: every user on the system gets the owner's full power. A world-writable script that a higher-privileged user runs lets an attacker plant their own commands. Hunting for such files is a standard privilege escalation step. Give the minimum permissions needed.

7. Ownership and sudo

Every file has an owner and a group (the two names in `ls -l`). `chown` changes **who owns** a file, permanently. It does not change the permissions. `chmod` changes permissions.

`sudo` runs **one command** as root, temporarily. Nothing about the file changes. It works for me because I am in the `sudo` group. Think of borrowing root's keys for one task and handing them straight back.

```
cat wazuh-install-files.tar      # Permission denied
sudo cat wazuh-install-files.tar  # works (but dumps binary garbage)
```

Checks: `whoami` (my name), `groups` (my groups), `id` (user ID: root is 0, mine is 1000).

8. The user files

File	What it holds
<code>/etc/passwd</code>	One line per account, 7 fields separated by colons. Service accounts often use <code>no login</code> . Readable by everyone.
<code>/etc/shadow</code>	Password hashes. Permissions <code>rw-r-----</code> , group <code>shadow</code> , so I get "Permission denied" without <code>sudo</code> .

9. grep and pipes

```
grep "word" file.txt      # lines containing word
grep -w word file.txt     # whole word only
grep -v word file.txt     # lines that do NOT match
ls -l /etc | grep "pass"  # pipe: output of ls becomes input of grep
```

The pipe `|` feeds one command's output into the next, so commands can work without a filename.

10. Processes

```
ps aux                    # every running process, each with a PID
ps aux | grep zsh | grep -v grep  # find zsh, hide grep matching itself
```

The first grep finds zsh, but the grep command itself also contains the word, so `grep -v grep` removes that extra line.

11. Services with systemctl

```
systemctl status NetworkManager
systemctl is-enabled wazuh-manager
systemctl is-active wazuh-manager
```

Word	Question it answers
active (running) / inactive (dead)	Is it running right now ?
enabled / disabled	Does it start at boot ?

The two are independent. NetworkManager: enabled and active (running), and my network works. Wazuh: disabled and inactive (dead), so it is not running and will not start by itself. Press q to leave the pager when status output shows (END). Do not restart Wazuh until I can read its logs.

12. Things I got wrong (revisit these)

- ~ is home, not the current folder (that is .).
- x on a folder means enter it, not run it.
- Numeric chmod digits are written side by side, not added together.
- sudo is temporary power for one command, not a permanent permission change.
- The "Active" word and the "enabled" word describe different things.

13. Cheat sheet

Task	Command
Where am I	<code>pwd</code>
Go home	<code>cd ~</code>
Detailed list with hidden files	<code>ls -la</code>
Copy / move / rename	<code>cp / mv</code>
Delete empty folder / folder with contents	<code>rmdir / rm -r (careful)</code>
Find a file by name	<code>find ~ -name "x"</code>
Make a script runnable	<code>chmod u+x script.sh</code>
Run one command as root	<code>sudo command</code>
Filter output	<code>command grep word</code>
Is a service running / at boot	<code>systemctl is-active / is-enabled</code>

Next on the roadmap

Windows fundamentals (CMD, PowerShell, users, services, registry), then networking fundamentals before Nmap and Wireshark. Still to practice from Linux: `cp -r`, `less`, `wc -l`, `sort`, `head/tail`, SSH

basics, and a first Bash script.